

ONIT-PNG : Rapport d'Audit Complet

Observatoire National Intelligent des Telecommunications - Republique de Guinee
(ARPT)

1. Resume Executif	3
1.1 Synthese des Scores	3
1.2 Decompte des Problemes	3
2. Audit Backend	4
2.1 Routes API - Statut par Route	4
2.2 Bug Critique : RLS Regionnal Cass	5
2.3 Systeme d'Authentification	5
2.4 Tests de Securite - Resultats	6
2.5 Base de Donnees	6
2.6 Controle d'Acces Base sur les Roles (RBAC)	7
2.7 Donnees Fictives en Dur	8
3. Audit Frontend	9
3.1 Statut des Composants	9
3.2 Accessibilite (Score : 2/10)	9
3.3 Flux Utilisateur et Navigation	10
4. Audit Production	11
4.1 Configuration Docker	11
4.2 Vulnerabilites de Securite	11
4.3 Performance et Scalabilite	12

4.4 Monitoring et Observabilite	13
5. Plan de Correction Priorise	13
5.1 Phase 1 : Securite Critique (2 semaines)	13
5.2 Phase 2 : Migration Base de Donnees (2 semaines)	14
5.3 Phase 3 : Fiabilite et Monitoring (1 semaine)	14
5.4 Phases 4-5 : CI/CD et Scalabilite (4-6 semaines)	15
6. Donnees Fonctionnelles Verifiees	16
6.1 Contenu de la Base de Donnees	16
6.2 Resultats des Tests API	16
7. Recommandations Finales	17
7.1 Actions Immédiatees (cette semaine)	17
7.2 Architecture Cible pour la Production	18
7.3 Points Positifs	18

1. Resume Executif

Ce rapport presente les resultats de l'audit complet de la plateforme ONIT-PNG (Observatoire National Intelligent des Telecommunications de la Republique de Guinee), developpee pour le compte de l'ARPT (Autorite de Regulation des Postes et Telecommunications). L'audit couvre l'ensemble des composantes backend (API, base de donnees, authentication, RBAC), frontend (composants React, UX, accessibilite) et la preparation au deploiement en production (Docker, securite, performance, monitoring).

L'audit a ete realise par analyse statique du code source, tests fonctionnels des endpoints API avec authentication reelle, verification de la base de donnees SQLite via Prisma, et evaluation de l'infrastructure Docker. Chaque route API a ete testee individuellement, les mecanismes RBAC ont ete verifies avec differents roles utilisateur, et les vulnerabilites de securite ont ete confirmees par des tests d'intrusion reussis (creation d'alertes sans authentication, injection XSS stockee).

1.1 Synthese des Scores

Domaine	Score	Statut
Backend (API + Auth + DB)	38/100	Non operationnel
Frontend (UI + UX + Accessibilite)	62/100	Partiel
Production (Docker + Securite + Perf)	28/100	Non pret
SCORE GLOBAL	43/100	Prototype

Tableau 1 : Synthese des scores d'audit par domaine

Conclusion principale : La plateforme ONIT-PNG est un prototype fonctionnel qui demontre les fonctionnalites attendues, mais elle presente des lacunes critiques en matiere de securite (secrets en clair, pas de HTTPS, creation d'alertes sans authentication), des bugs dans le controle d'accès regional (RLS), et des donnees fictives en dur dans les tableaux de bord. Le systeme ne peut pas etre deploye en production dans son etat actuel sans corrections majeures.

1.2 Decompte des Problemes

Severite	Backend	Frontend	Production	Total
CRITIQUE	8	5	9	22

HAUTE	7	7	9	23
MOYENNE	6	8	6	20
BASSE	5	10	4	19

Tableau 2 : Decompte des problemes par severite et domaine

2. Audit Backend

L'audit backend couvre les 14 routes API, le systeme d'authentification NextAuth v4, le modele de donnees Prisma avec SQLite, le systeme RBAC (Role-Based Access Control), et les politiques de restriction d'accès aux données (Data Access Policies). Chaque route a été testée avec et sans authentification pour vérifier les contrôles d'accès.

2.1 Routes API - Statut par Route

Route	Methode(s)	Auth	Validation	RLS	Statut
/api/auth/[...nextauth]	GET/POST	N/A	Basique	N/A	Partiel
/api/dashboard	GET	Aucune	Aucune	Bug	Partiel
/api/qos	GET	OK	Aucune	Bug	Partiel
/api/scoring	GET	Public	Aucune	OK	Partiel
/api/alerts	GET/POST /PATCH	Mixte	Aucune	Bug	Bug
/api/users	GET/POST /PATCH	OK	Minimale	N/A	Partiel
/api/campaigns	GET/POST	OK	Aucune	Bug	Partiel
/api/mesures	GET/POST /PUT	OK	Bonne	Bug	OK
/api/roles	GET	OK	N/A	N/A	Partiel
/api/audit-logs	GET	OK	Aucune	N/A	Partiel
/api/map	GET	Public	Aucune	Bug	Partiel
/api/scores	GET/POST	Mixte	POST OK	OK	Partiel
/api/reports	GET/POST	Mixte	Aucune	OK	Partiel
/api (root)	GET	N/A	N/A	N/A	OK

Tableau 3 : Statut detaille de chaque route API

2.2 Bug Critique : RLS Regional Cass

Le bug le plus impactant du backend est la rupture complete du systeme de restriction d'accès aux données par region (Row-Level Security). La fonction `getAccessibleRegions()` dans `rbac.ts` retourne les codes de regions (par exemple ["CON", "KIN", "BOK"]) au lieu de leurs identifiants CUID (par exemple "cmp7hylqn003xpy04xco2sapj"). Or, toutes les requêtes Prisma utilisent ces valeurs dans des filtres `regionId: { in: accessibleRegIds }` ou `regionId` attend un CUID, pas un code.

Impact : Les utilisateurs avec un scope "own_region" (INGENIEUR_RF) ne voient aucune donnée car la comparaison `regionId IN ["CON"]` ne correspond à aucun enregistrement en base. Ce bug affecte 7 routes sur 14 (dashboard, qos, alerts, campaigns, mesures, map, scoring). Le correctif est simple : remplacer `code: true` par `id: true` dans la requête de `getAccessibleRegions()`.

Champ	Valeur retournée (BUG)	Valeur attendue
regionId en base	cmp7hylqn003xpy04xco2sapj	CUID genere par Prisma
<code>getAccessibleRegions()</code>	["CON", "KIN", "BOK", ...]	["cmp7hylqn003x...", "cmp7hylqo003y...", ...]
Correspondance	0 resultat	Donnees filtrees

Tableau 4 : Preuve du bug RLS - codes vs identifiants

2.3 Systeme d'Authentification

Le systeme d'authentification repose sur NextAuth v4 avec le provider Credentials et une strategie JWT (duree de vie 8 heures). Les mots de passe sont compares via `bcryptjs`, ce qui est correct. Cependant, le systeme presente des vulnerabilites critiques qui le rendent impropre a un deploiement en production. L'audit fonctionnel a confirme que la connexion fonctionne correctement avec les identifiants `admin@arpt.gn / Admin@2026!`, que le JWT contient bien le role et les permissions, et que la session est correctement creee avec le cookie `next-auth.session-token`.

Vulnerabilite	Severite	Detail
---------------	----------	--------

NEXTAUTH_SECRET code en dur	CRITIQUE	Valeur "onit-png-secret-key-2026-guinee" dans le source - permet de forger des JWT
Pas de limitation de debit (rate limiting)	CRITIQUE	Le endpoint de login n'a aucune protection contre les attaques par force brute
Pas de verrouillage de compte	CRITIQUE	Aucun blocage apres N tentatives echouees
Cookies non securises (HTTP)	HAUTE	useSecureCookies=false, secure=false - jeton transmissible en clair
Pas de politique de mot de passe	HAUTE	Aucune exigence de longueur, complexite ou rotation
Mots de passe dans les logs	MOYENNE	console.log des emails a chaque connexion
Pas d'invalidation de session	MOYENNE	JWT sans mecanisme de revocation cote serveur
Pas de 2FA/MFA	MOYENNE	Authentification a un seul facteur uniquement

Tableau 5 : Vulnerabilites du systeme d'authentification

2.4 Tests de Securite - Resultats

Des tests d'intrusion ont ete executes contre l'application en cours d'execution pour confirmer les vulnerabilites identifiees par l'analyse statique. Voici les resultats des tests reussis (c'est-a-dire les attaques qui ont fonctionNE, confirmant la vulnerabilite) :

Test	Resultat	Preuve
Creation d'alerte sans authentication	VULNERABLE	POST /api/alerts retourne HTTP 200 avec ID de l'alerte creee
Injection XSS dans les alertes	VULNERABLE	Le script <script>alert(1)</script> est stocke tel quel dans le champ message
Acces non authentifie au dashboard	VULNERABLE	GET /api/dashboard retourne toutes les KPIs sans session
Acces operateur aux users/roles	PROTEGE	Retourne 401 "Non autorise" pour les roles non-admin
Connexion avec identifiants valides	FONCTIONNE	Session JWT creee avec role et permissions corrects
Acces map/scoring sans auth	PUBLIC	Donnees geographiques et scores accessibles sans session

Tableau 6 : Resultats des tests d'intrusion

2.5 Base de Donnees

La base de donnees utilise SQLite via Prisma ORM. Le schema est bien structure avec 12 modeles couvrant les utilisateurs, roles, permissions, mesures QoS, alertes, campagnes, scores operateurs, rapports, et journaux d'audit. Le seed contient 10 utilisateurs, 9 roles, 94 permissions, 78 mesures QoS, 8 campagnes et 8 regions administratives de Guinee. Cependant, SQLite presente des limites majeures pour un usage en production : verrouillage en ecriture unique, pas de concurrence, pas d'index definis dans le schema Prisma, et chemins de base de donnees incoherents entre les differents fichiers de configuration (4 chemins differents repertories).

Probleme	Severite	Impact
SQLite en production	CRITIQUE	Pas de concurrence en ecriture, un seul thread d'ecriture a la fois
Pas d'index en base	HAUTE	Full table scan sur chaque requete filteree (operateurId, regionId, timestamp)
Pas de migrations Prisma	CRITIQUE	Utilisation de db push au lieu de migrate deploy - pas d'historique versionne
Chemins DB incoherents	HAUTE	4 chemins differents entre .env, docker-compose, start.sh, et .env.example
Enums stockes en strings	MOYENNE	statut, type, severity, format sans validation - valeurs arbitraires possibles
Rapport.generePar est String	MOYENNE	Pas de relation vers User - integrite referentielle brisee

Tableau 7 : Problemes de la base de donnees

2.6 Controle d'Acces Base sur les Roles (RBAC)

Le systeme RBAC definit 9 roles avec 94 permissions reparties sur 4 ressources principales (campaigns, reports, scores, alerts). Le modele de donnees inclut des DataAccessPolicies qui definissent le scope d'accès (all, own_org, own_region, public_only) par role et ressource. L'audit a revele que le RBAC fonctionne correctement pour les controles de permission (checkPermission), mais que le filtrage par region est completement casse par le bug RLS decrit precedemment. De plus, certaines routes (dashboard, scoring, map) ne verifient pas du tout les permissions et exposent les donnees a tous les utilisateurs authentifies.

Role	Scope Campagnes	Scope Scores	Regions	Test
SUPER_ADMIN	all	all	Toutes	OK

DG	all	N/A	Toutes	OK
DGA	all	N/A	Toutes	OK
DIRECTEUR_TECH	all	N/A	Toutes	OK
INGENIEUR_RF	own_region	N/A	Bug	0 donnee
ANALYSTE_QOS	all	N/A	Toutes	OK
AUDITEUR	all	N/A	Toutes	OK
OPERATEUR	own_org	own_org	Organisation	Partiel
PUBLIC	public_only	public_only	Publique	Partiel

Tableau 8 : Matrice RBAC et resultats des tests par role

2.7 Donnees Fictives en Dur

L'audit a identifie de nombreuses valeurs fictives codees en dur dans les routes API, ce qui signifie que les tableaux de bord affichent des donnees qui ne proviennent pas de la base de donnees mais de calculs arbitraires ou de constantes. Cela mine la credibilite du systeme et peut induire en erreur les decideurs. Les valeurs suivantes ont ete trouvees dans le code source de `/api/dashboard/route.ts` et confirmees par les tests fonctionnels :

Valeur	Source	Fichier:Ligne	Impact
couvertureNational e = 67	Constante par default	dashboard.ts:3 1	Affiche 100% si donnee presente, 67% sinon
scoreQosGlobal = 72	Constante par default	dashboard.ts:3 7	Score fictif si pas de donnee
trend = 2.3 / -1.2 / -12 / 340	Valeurs en dur	dashboard.ts:1 30-133	Tendances non calculees depuis les donnees reelles
zonesBlanches + 200	Offset arbitrair e	dashboard.ts:1 32	Ajoute 200 zones blanches au compte reel
slaCompliance.glo bal = 84	En dur	dashboard.ts:1 39	SLA jamais calcule depuis les donnees
innovation = QoS * 0.9	Calcul fictif	dashboard.ts:7 8	Sous-score non base sur des metriques reelles
Math.random() pour taille rapport	Aleatoire	reports.ts:25	Taille differente a chaque requete

Tableau 9 : Donnees fictives identifiees dans les routes API

3. Audit Frontend

L'audit frontend couvre 24 composants React, le systeme de navigation par onglets, les flux utilisateur (connexion, deconnexion, navigation), l'accessibilite WCAG, la responsivite, et la qualite des visualisations de donnees. L'application est une SPA (Single Page Application) avec un systeme d'onglets geres par l'etat client, sans routage URL.

3.1 Statut des Composants

Composant	Lignes	Statut	Donnees Reelles	Problemes
page.tsx	72	Complet	Oui	Pas d'error boundary
onit-layout.tsx	339	Complet	Partiel	N PATCH paralleles pour mark-all-read
login-modal.tsx	216	Complet	N/A	Mot de passe en clair dans le code
dashboard-dg.tsx	248	Complet	Partiel	Donnees de fallback sans indication
dashboard-qos.tsx	204	Complet	Partiel	Pas de debounce sur les filtres
dashboard-sig.tsx	169	Complet	Partiel	Drive tests non implements
dashboard-scoring.tsx	167	Complet	Partiel	Recommandations en dur
dashboard-audit.tsx	135	Partiel	Non	Resultats + Drive Test fictifs
dashboard-cyber.tsx	151	Partiel	Non	100% de donnees fictives
dashboard-public.tsx	240	Complet	Partiel	4 appels API paralleles sans loader
dashboard-admin.tsx	673	Complet	Partiel	Pas d'edition utilisateur
dashboard-reports.tsx	226	Complet	Partiel	Telechargement factice
guinea-map-leaflet.tsx	100	Complet	Oui	Key prop force le remontage
mini-chart.tsx	362	Complet	Oui	Pas de tooltips, pas de memo

Tableau 10 : Statut detaille des composants frontend

3.2 Accessibilite (Score : 2/10)

L'accessibilite est le domaine le plus defaillant du frontend. L'application ne respecte pratiquement aucune directive WCAG 2.1 AA. Les problemes les plus critiques incluent l'absence de role dialog et aria-modal sur la modale de connexion, l'absence de piege de focus (focus trap) permettant de naviguer hors de la modale avec Tab, l'absence d'attributs aria-label sur les boutons iconiques, et des problemes de contraste de couleurs (text-slate-500 sur fond sombre donne un ratio de contraste d'environ 3.2:1, bien en dessous du minimum requis de 4.5:1).

Critere WCAG	Statut	Impact
Focus trap dans les modales	Absent	Navigation clavier hors modale possible
aria-label sur boutons icones	Absent	Lecteurs d'ecran ne peuvent pas identifier les actions
Contraste texte/bg (AA)	Non conforme	Texte illisible pour les utilisateurs malvoyants
Skip navigation link	Absent	Navigation clavier obligatoire a travers toute la sidebar
aria-live pour contenu dynamique	Absent	Alertes et notifications non annoncees
Keyboard navigation (Escape)	Absent	Impossible de fermer modales/dropdowns au clavier
SVG charts accessibles	Absent	Graphiques invisibles pour les lecteurs d'ecran

Tableau 11 : Problemes d'accessibilite critiques

3.3 Flux Utilisateur et Navigation

L'application utilise un systeme de navigation par onglets geres uniquement par l'etat client (useState), sans routage URL. Cela signifie qu'il est impossible de creer un lien profond vers un tableau de bord specifique, le bouton retour du navigateur ne navigue pas entre les onglets, et l'etat de l'onglet est perdu au rechargement de la page. La barre de recherche dans l'en-tete est purement decorative sans fonctionnalite de recherche. Apres la connexion, l'application effectue un rechargement complet de la page (window.location.href = '/') au lieu d'une navigation cote client, ce qui detruit tout l'etat React. Le menu utilisateur propose des boutons "Changer le mot de passe" et "Journal d'audit" qui n'ont aucun gestionnaire de clic.

4. Audit Production

L'audit de preparation a la production evalue l'infrastructure Docker, la configuration de securite, les performances, la scalabilite, le monitoring, les sauvegardes, et les pipelines CI/CD. Le score global de 28/100 reflete l'etat actuel de prototype qui necessite des investissements significatifs avant de pouvoir etre expose sur Internet.

4.1 Configuration Docker

Le Dockerfile utilise un build multi-stage (deps, builder, runner) avec une image de base node:20-alpine, un utilisateur non-root (nextjs:nodejs), et un health check. Le docker-compose.yml definit un volume persistant pour la base de donnees et un reseau bridge. Cependant, le build Docker echouera en production car l'etape deps utilise npm ci --only=production mais l'etape builder a besoin des dependances de developpement pour prisma generate et next build. De plus, le script CMD execute prisma db push a chaque demarrage, ce qui est dangereux en production.

Aspect	Statut	Detail
Multi-stage build	OK	3 etapes: deps, builder, runner
Utilisateur non-root	OK	nextjs:nodejs (uid/gid 1001)
Health check	OK	HTTP check sur /api/auth/session
Volume persistant	OK	onit-db:/app/db
Build deps stage	Bug	--only=production mais builder a besoin des devDeps
prisma db push au demarrage	Dangereux	Migration de schema a chaque demarrage de conteneur
Secrets en dur	Critique	NEXTAUTH_SECRET en clair dans docker-compose.yml
Pas de limites ressources	Manquant	Pas de mem_limit ou cpus definis

Tableau 12 : Evaluation de la configuration Docker

4.2 Vulnerabilites de Securite

L'audit de securite a identifie 21 vulnerabilites reparties en 7 critiques, 8 hautes et 6 moyennes. La vulnerabilite la plus grave est la presence du NEXTAUTH_SECRET dans le code source, ce qui permet a quiconque ayant acces

au depot de forger des jetons JWT valides et d'accéder a n'importe quel compte, ycompris SUPER_ADMIN. La seconde vulnerabilite critique est l'absence totale deHTTPS, avec les cookies de session transmis en clair sur le reseau. Le Caddyfilecontient egalement une vulnerabilite SSRF via le parametre XTransformPort qui permet de proxyer des requetes vers n'importe quel port interne.

ID	Vulnerabilite	Severite
S1	NEXTAUTH_SECRET code en dur dans 4 fichiers (.env, docker-compose, start.sh, start-standalone.sh)	CRITIQUE
S2	SSRF via XTransformPort dans Caddyfile - proxy vers n'importe quel port interne	CRITIQUE
S3	Pas de HTTPS/TLS - tous les cookies de session transmis en clair	CRITIQUE
S4	Cookies non securises (secure: false) - vol de session possible	CRITIQUE
S5	ignoreBuildErrors:true - erreurs TypeScript ignorees en production	CRITIQUE
S6	Secret fallback dans le code source - si NEXTAUTH_SECRET absent, valeur connue utilisee	CRITIQUE
S7	Identifiants par defaut dans le README (admin@arpt.gn / Admin@2026!)	CRITIQUE
S8	Pas de rate limiting sur le login - force brute possible	HAUTE
S9	Pas de verrouillage de compte apres tentatives echouees	HAUTE
S10	Pas de validation des entrees (Zod) sur les routes POST/PATCH	HAUTE
S11	Pas de middleware Next.js - pas de CSRF, CORS ou filtrage centralise	HAUTE

Tableau 13 : Vulnerabilites de securite (extrait des 21 identifiees)

4.3 Performance et Scalabilite

Les performances de l'application sont limitees par plusieurs facteurs structurels. La base de donnees SQLite ne supporte qu'un seul processus d'ecriture a la fois, ce qui signifie que sous charge, toutes les ecritures seront mises en file d'attente. L'API dashboard charge l'ensemble des mesures en memoire avant de les filtrer (pattern N+1), ce qui provoquera des crashes par manque de memoire lorsque le volume de donnees augmentera. Il n'y a aucune couche de cache (ni Redis, ni cache en memoire, ni ISR Next.js), donc chaque chargement de tableau de bord atteint la base de donnees directement. L'application ne peut pas etre horizontalement scalable car SQLite ne peut pas etre partage entre plusieurs instances.

Goulot d'etirement	Severite	Plafond actuel
SQLite ecriture unique	CRITIQUE	1-5 ecritures concurrentes max
Pas de cache	HAUTE	Chaque requete atteint la DB
Chargement de toutes les mesures en memoire	HAUTE	Crash OOM au-dela de ~10000 mesures
Pas de pagination (dashboard, map)	HAUTE	Toutes les donnees envoyees en une fois
Pas d'index en base	HAUTE	Full table scan sur chaque requete filtree
Pas de scalabilite horizontale	HAUTE	1 conteneur maximum (SQLite non partageable)

Tableau 14 : Goulots d'etirement de performance

4.4 Monitoring et Observabilite

L'observabilite de l'application est quasiment inexistante. Les seuls logs sont des console.log et console.error sans structure, niveaux ni contexte. Il n'y a pas de service de suivi des erreurs (Sentry), pas de metriques applicatives (Prometheus), pas de tracing distribue, et pas de systeme d'alerting. Le health check Docker verifie uniquement que le serveur HTTP repond, sans verifier la connectivite a la base de donnees. Il n'y a pas de strategie de sauvegarde automatisee pour la base SQLite, et copier un fichier SQLite en cours d'utilisation peut corrompre la sauvegarde. Aucun pipeline CI/CD n'est configure.

5. Plan de Correction Priorise

Le plan de correction est organise en 5 phases prioritaires, estimees pour un developpeur travaillant a temps plein. Les deux premieres phases sont indispensables avant tout deploiement, meme interne. Les phases suivantes peuvent etre realisees de maniere progressive.

5.1 Phase 1 : Securite Critique (2 semaines)

Action	Effort	Impact
Corriger getAccessibleRegions() : retourner les IDs au lieu des codes	1 heure	Corrige le RLS sur 7 routes

Supprimer les secrets du code source, utiliser des variables d'environnement ou Docker secrets	4 heures	Empeche le forgeage de JWT
Activer HTTPS avec Caddy + domaine	4 heures	Protege les cookies de session
Ajouter un rate limiting sur le login	4 heures	Empeche les attaques par force brute
Ajouter la validation Zod sur toutes les routes POST/PATCH	2 jours	Empeche les injections XSS et les donnees corrompues
Exiger l'authentification pour POST /api/alerts	2 heures	Arrete la creation d'alertes non autorisees
Supprimer ignoreBuildErrors: true dans next.config.ts	1 jour	Empeche le deploiement de code casse
Supprimer le SSRF du Caddyfile (XTransformPort)	30 min	Empeche l'accès aux services internes

Tableau 15 : Phase 1 - Corrections de securite critiques

5.2 Phase 2 : Migration Base de Donnees (2 semaines)

Action	Effort	Impact
Migrer de SQLite vers PostgreSQL (Supabase, Neon, ou RDS)	3 jours	Supporte la concurrence et la scalabilite
Configurer prisma migrate deploy pour les migrations	1 jour	Migrations versionnees et sures
Ajouter des index sur operateurId, regionId, campagneId, timestamp	2 heures	Requetes 10-100x plus rapides
Corriger les chemins DATABASE_URL incoherents	1 heure	Consistance entre environnements
Remplacer les enums en string par de vrais enums Prisma	2 jours	Validation des valeurs au niveau DB
Ajouter une strategie de sauvegarde automatisee (pg_dump)	1 jour	Recovery en cas de perte de donnees

Tableau 16 : Phase 2 - Migration base de donnees

5.3 Phase 3 : Fiabilite et Monitoring (1 semaine)

Action	Effort	Impact
Ajouter un logging structure (Pino ou Winston)	2 jours	Debugging et audit en production

Integrer Sentry pour le suivi des erreurs	1 jour	Detection proactive des bugs
Ajouter un health check qui verifie la DB	2 heures	Detection des pannes de base
Ajouter des limites de ressources Docker (mem_limit, cpus)	1 heure	Empeche les crashes OOM
Ajouter une Error Boundary React	2 heures	Empeche les ecrans blancs
Remplacer les donnees fictives par des calculs reels	3 jours	Donnees credibles dans les dashboards

Tableau 17 : Phase 3 - Fiabilite et monitoring

5.4 Phases 4-5 : CI/CD et Scalabilite (4-6 semaines)

Phase 4 - CI/CD et Tests (2 semaines) : Configurer GitHub Actions pour le lint, le type-check, les tests unitaires et le build automatique. Ecrire des tests unitaires pour le RBAC, des tests d'integration pour les routes API, et des tests E2E pour les flux critiques (connexion, import de donnees, generation de rapports). Reactiver progressivement les regles ESLint desactivees et activer noImplicitAny dans tsconfig.json.

Phase 5 - Scalabilite (2-3 semaines) : Ajouter Redis pour le cache des tableaux de bord et le rate limiting. Implementer la pagination curseur sur tous les endpoints liste. Configurer Docker Swarm ou Kubernetes pour la scalabilite horizontale. Ajouter un CDN (Cloudflare) pour les assets statiques et la protection DDoS. Implementer les Server-Sent Events ou WebSocket pour les mises a jour en temps reel des tableaux de bord.

Phase	Duree Estimee	Prerequis	Priorite
Phase 1 : Securite	2 semaines	Nom de domaine, certificat SSL	CRITIQUE
Phase 2 : BDD	2 semaines	Hebergement PostgreSQL	CRITIQUE
Phase 3 : Fiabilite	1 semaine	Compte Sentry	HAUTE
Phase 4 : CI/CD + Tests	2 semaines	GitHub Actions	MOYENNE
Phase 5 : Scalabilite	2-3 semaines	Redis, orchestration	BASSE
TOTAL	9-12 semaines		

Tableau 18 : Resume de l'effort par phase

6. Donnees Fonctionnelles Verifiees

Cette section presente les resultats des tests fonctionnels realises contre l'application en cours d'execution. Le serveur a ete lance en mode production (NODE_ENV=production) avec le build standalone Next.js, et les endpoints API ont ete testes avec curl.

6.1 Contenu de la Base de Donnees

Table	Enregistrements	Commentaire
User	10	9 roles couverts, tous actifs, meme mot de passe
Role	9	SUPER_ADMIN, DG, DGA, DIRECTEUR_TECH, INGENIEUR_RF, ANALYSTE_QOS, AUDITEUR, OPERATEUR_READONLY, PUBLIC
Permission	94	Reparties sur 4 ressources principales
MesureQoS	78	3 operateurs x 8 regions, donnees de test
Alerte	14	Inclut les alertes de test d'intrusion
Campagne	8	Campagnes de test drive/walk
ScoreOperateur	12	4 scores par operateur (3 operateurs)
Rapport	8	Rapports de test multi-formats
AuditLog	19	Trace des actions effectuees
Region	8	Conakry, Kindia, Boké, Labé, Mamou, Faranah, Kankan, N'Zérékoré
Operateur	3	Orange Guinée, MTN Guinée, Celcom Guinée
DataAccessPolicy	18	Policies RLS par role et ressource

Tableau 19 : Contenu verifie de la base de donnees

6.2 Resultats des Tests API

Endpoint	Auth	HTTP	Resultat
GET /api	Non	200	Health check OK
POST /api/auth/callback/credentials	Non	302	Connexion reussie, JWT cree
GET /api/auth/session	Cookie	200	Session avec email + role

GET /api/dashboard	Non	200	KPIs, operateurs, alertes, SLA
GET /api/qos	Oui	200/401	Metriques + benchmark + heatmap
GET /api/alerts	Oui	200/401	Liste des alertes
POST /api/alerts	Non	200	Alerte creee sans auth - VULNERABLE
GET /api/users	Admin	200/401	Liste utilisateurs (admin seulement)
GET /api/roles	Admin	200/401	9 roles avec permissions
GET /api/campaigns	Oui	200/401	8 campagnes
GET /api/map	Non	200	8 regions, 78 points, 3 operateurs
GET /api/scoring	Non	200	3 operateurs, radar data
GET /api/reports	Cookie	200	8 rapports

Tableau 20 : Resultats des tests API fonctionnels

7. Recommandations Finales

Base sur l'ensemble de l'audit, voici les recommandations finales classees par urgence. Il est imperatif de ne pas deployer cette application sur un serveur accessible publiquement avant que les correctifs de la Phase 1 ne soient appliques. Un deploiement interne (intranet ARPT) est possible apres les Phases 1 et 2, tandis qu'un deploiement Internet requiert les Phases 1 a 3.

7.1 Actions Immédiates (cette semaine)

1. Corriger le bug RLS dans rbac.ts : remplacer code: true par id: true dans getAccessibleRegions(). C'est un correctif d'une ligne qui corrige 7 routes simultanement et permet aux ingenieurs RF de voir leurs donnees regionales. C'est le bug le plus impactant et le plus simple a corriger.
2. Supprimer les secrets du code source. Generer un NEXTAUTH_SECRET cryptographiquement aleatoire (minimum 32 octets), le stocker dans .env.local (pas dans .env), et l'injecter via Docker secrets ou des variables d'environnement en production. Supprimer la valeur de fallback dans auth route.ts.

3. Exiger l'authentification pour POST /api/alerts. Ce point d'accès permet actuellement à n'importe qui de créer des alertes dans le système sans authentification, y compris avec du contenu XSS. Ajouter une vérification de session et un CAPTCHA pour les signalements publics.
4. Supprimer le SSRF du Caddyfile. Supprimer entièrement le bloc XTransformPort qui permet de proxyer des requêtes vers des ports internes arbitraires. C'est une vulnérabilité critique qui peut être exploitée pour accéder à des services internes.

7.2 Architecture Cible pour la Production

L'architecture cible recommandée pour un déploiement en production comprend les composants suivants : Cloudflare en frontal pour le CDN et la protection DDoS, Caddy comme reverse proxy avec HTTPS automatique via Let's Encrypt, 2 à 3 instances Next.js sans état derrière un load balancer, PostgreSQL comme base de données (service managé tel que Supabase ou Neon recommandé), Redis pour le cache des tableaux de bord et le rate limiting, et un système de monitoring avec Sentry pour les erreurs, Prometheus pour les métriques, et des logs structurés avec Pino. L'estimation totale pour atteindre ce niveau de maturité est de 9 à 12 semaines pour un développeur, ou 5 à 7 semaines pour une équipe de 2 développeurs.

7.3 Points Positifs

Malgré les problèmes identifiés, la plateforme ONIT-PNG présente plusieurs qualités notables. Le design visuel est professionnel et cohérent (mode sombre bleu nuit + or), les 9 tableaux de bord couvrent efficacement les différents besoins métier (DG, QoS, SIG, Scoring, Audit, Cybersecurite, Administration, Rapports, Portail Public), la carte Leaflet avec les 8 régions de Guinée est fonctionnelle et précise, le modèle de données Prisma est bien structuré avec un bon support RBAC, le seed de données est complet et réaliste, et l'interface utilisateur avec shadcn/ui est moderne et répond aux standards actuels du développement web. Ces fondations solides permettent d'envisager une mise en production réussie après les corrections nécessaires.